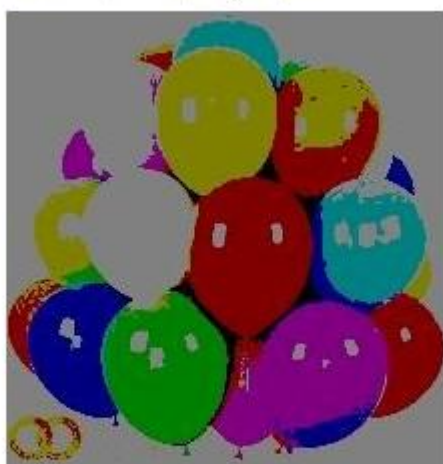






```

# 功能按鈕
tk.Label(self.root, text="選擇量化算法:", font=('Arial', 10, 'bold')).place(x=10, y=180)

# (1) 教科書之作法 (L-1 levels)
tk.Button(self.root, text="1. 教科書作法 (L-1)", command=lambda: self.process_quantization(1), **btn_style).place(x=10, y=210)

# (2) 每一層取最小值
tk.Button(self.root, text="2. 取最小值 (Min)", command=lambda: self.process_quantization(2), **btn_style).place(x=10, y=250)

# (3) 每一層取中間值
tk.Button(self.root, text="3. 取中間值 (Mid)", command=lambda: self.process_quantization(3), **btn_style).place(x=10, y=290)

# 顯示視窗
self.l1 = tk.Label(self.root, text="Original Image")
self.l1.place(x=300, y=10)
self.l2 = tk.Label(self.root, text="Processed Image")
self.l2.place(x=780, y=10)

```

```

# 功能按鈕
tk.Label(self.root, text="選擇量化算法:", font=('Arial', 10, 'bold')).place(x=10, y=180)

# (1) 教科書之作法 (L-1 levels)
tk.Button(self.root, text="1. 教科書作法 (L-1)", command=lambda: self.process_quantization(1), **btn_style).place(x=10, y=210)

# (2) 每一層取最小值
tk.Button(self.root, text="2. 取最小值 (Min)", command=lambda: self.process_quantization(2), **btn_style).place(x=10, y=250)

# (3) 每一層取中間值
tk.Button(self.root, text="3. 取中間值 (Mid)", command=lambda: self.process_quantization(3), **btn_style).place(x=10, y=290)

# 顯示視窗
self.l1 = tk.Label(self.root, text="Original Image")
self.l1.place(x=300, y=10)
self.l2 = tk.Label(self.root, text="Processed Image")
self.l2.place(x=780, y=10)

```

```

def load_image(self):
    path = filedialog.askopenfilename()
    if path:
        img_array = np.fromfile(path, np.uint8)
        self.original_img = cv2.imdecode(img_array, cv2.IMREAD_COLOR)

        if self.original_img is not None:
            self.original_img = cv2.cvtColor(self.original_img, cv2.COLOR_BGR2RGB)
            self.display_image(self.original_img, self.l1)
        else:
            messagebox.showerror("錯誤", "無法讀取該圖片，請確認格式。")

def get_bits(self):
    (variable) bits_var: StringVar
    try:
        b = int(self.bits_var.get())
        if b <= 0 or b > 8: raise ValueError
        return b
    except:
        messagebox.showerror("錯誤", "請輸入 1 到 8 之間的整數")
        return None

```

```

def process_quantization(self, method):
    if self.original_img is None:
        messagebox.showwarning("警告", "請先選擇圖片")
        return

    bits = self.get_bits()
    if bits is None: return

    L = 2 ** bits # 量化階數
    img = self.original_img.astype(np.float32)

    if method == 1:
        # 教科書作法: 將 0-255 切成 L-1 個間隔
        # 公式:  $\text{round}(I / (255/(L-1))) * (255/(L-1))$ 
        interval = 255 / (L - 1)
        res = np.round(img / interval) * interval

    elif method == 2:
        # 每一層取最小值: 區間長度為 256/L
        # 公式:  $\text{floor}(I / (256/L)) * (256/L)$ 
        interval = 256 / L
        res = np.floor(img / interval) * interval

```

```

    elif method == 3:
        # 每一層取中間值: 最小值 + 半個區間長度
        # 公式:  $\text{floor}(I / (256/L)) * (256/L) + (\text{interval}/2)$ 
        interval = 256 / L
        res = np.floor(img / interval) * interval + (interval / 2)

    self.processed_img = np.clip(res, 0, 255).astype(np.uint8)
    self.display_image(self.processed_img, self.l2)

```

```

def display_image(self, img_array, label):
    h, w, _ = img_array.shape
    scale = min(450/w, 450/h)
    img_pil = Image.fromarray(img_array).resize((int(w*scale), int(h*scale)))
    img_tk = ImageTk.PhotoImage(img_pil)
    label.config(image=img_tk)
    label.image = img_tk

```

```

def save_image(self):
    if self.processed_img is not None:
        path = filedialog.asksaveasfilename(defaultextension=".jpg")
        if path:
            # 先轉回 BGR 顏色
            save_img = cv2.cvtColor(self.processed_img, cv2.COLOR_RGB2BGR)
            # 解決中文路徑問題的存法
            ext = path.split('.')[-1]
            res, img_encode = cv2.imencode(f'.{ext}', save_img)
            if res:
                img_encode.tofile(path)
                messagebox.showinfo("提示", "儲存成功")

```

```

if __name__ == "__main__":
    root = tk.Tk()
    app = ImageQuantizerApp(root)
    root.mainloop()

```

這次的作業主要是在做影像量化 (Image Quantization)，透過改變影像的 bits 數來觀察影像的變化。在實作的過程中，我把影像的位元數改成 1、2、4、6 bits，分別對應到不同的灰階層級數 (levels)。當 bits 越少時，影像能表示的灰階數就越少，所以畫面會變得比較粗糙，也會出現比較明顯的色塊或階梯感；當 bits 增加時，影像就會比較接近原始圖片，看起來也比較自然。

另外，這次作業還比較了三種不同的還原方式，包括教科書的方法、每個 level 取最小值，以及取中間值當代表值。實際比較之後可以發現，不同方法會讓影像呈現出不同的效果，其中取中間值的方式通常看起來會比較平均，也比較接近原本的影像。

透過這次作業，我對影像量化的概念有更清楚的理解，也學到影像在數位化過程中其實會因為資料量的限制而產生失真。整體來說，這個實作讓我更了解影像處理的一些基本原理。